Informe breve — TP02 Programación I

Grupo: 3
Comisión: .....

Diseño Top-Down aplicado

El problema "ejecutar cualquier misión" se descompuso en funciones con una responsabilidad clara cada una. comando_es_valido decide si un comando tiene formato correcto, sin tocar el robot, y se apoya en es_numero para revisar velocidad y tiempo. ejecutar_comando ejecuta un único comando ya validado y atrapa el ErrorDeSeguridad del robot si excede un límite físico. ejecutar_mision recorre la lista de comandos con un for, coordina a las dos funciones anteriores y usa continue para saltear los inválidos. generar_reporte calcula las estadísticas finales a partir del historial. Por último, correr_mision agrupa los pasos que se repiten para cada misión (crear el historial, ejecutar, reportar y detener el robot), crear_mi_mision devuelve la lista de comandos de la misión propia. Así, main() queda reducido a conectar el robot, llamar a correr_mision cuatro veces y desconectar.

Esta separación permitió probar cada parte por separado y ubicar rápido dónde estaba un problema; por ejemplo, distinguir si un comando se rechazaba por mal formato (comando_es_valido) o por exceder un límite del robot (ejecutar_comando). Además, como la cátedra no permite variables globales, cada función trabaja solo con sus parámetros y sus variables locales: los tipos válidos son locales a comando_es_valido y la misión propia es local a crear_mi_mision, que la devuelve con return.

Pasaje por valor y por referencia

Los valores numéricos (velocidad, tiempo) y los textos se comportan como pasaje por valor: si una función reasigna su parámetro, la variable original de quien la llamó no cambia. En cambio, historial es una lista, un objeto mutable, y se comporta como pasaje por referencia: ejecutar_mision hace historial.append(registro) sin devolver la lista, y ese cambio es visible en correr_mision apenas termina la llamada. Lo comprobé en la práctica: generar_reporte(historial), llamada justo después, ya tiene todos los registros agregados durante el recorrido, aunque ninguna función devolvió la lista. Como correr_mision crea un historial nuevo para cada misión, los reportes no se mezclan entre misiones.

¿Qué tuve que cambiar para ejecutar mi propia misión?

Ninguna función. Alcanzó con escribir una nueva lista de tuplas en crear_mi_mision() y pasarla a correr_mision, igual que las misiones de la cátedra. Lo mismo pasó con MISION_CUADRADO: se agregó con una sola línea en main(). Esto confirma el objetivo del diseño Top-Down: como cada función depende solo de sus parámetros y no de una misión escrita "a mano" en su interior, el programa acepta cualquier secuencia de comandos sin cambios estructurales.